

Phase 4

A Pipelined RV32I CPU in Verilog

EEL 4768 Computer Architecture
Fall 2026

Learning Objectives

By the end of this phase you should be able to:

- Build a 5-stage pipelined RV32I CPU in Verilog.
- Detect and resolve data hazards and control hazards.
- Implement forwarding logic to minimize pipeline stalls.

1 Submission

- **One submission per group.** You will manually submit your work to Webcourses as a zip file. Students in multiple groups, or not in a group by the time of submission, will receive **no credit** for the submission.
- **Files to submit:**

```
hart.v
alu.v
imm.v
rf.v
decoder.v
```

One module per file, and name the module the same as the filename. The testbenches instantiate modules named `hart`, `alu`, `imm`, `rf` and `decoder` by name.

2 Grading Rubric

Category	Points
Pipelined Processor	3.0
Hazard Detection	2.0
Forwarding Logic	2.0
Total	7.0

3 Verilog Coding Rules

Your submission is internally checked against a synthesizable subset. The rules listed below will result in a 0 for the specific file.

Rule	What it means for you
No <code>*</code> , <code>/</code> , <code>%</code> , <code>**</code>	Arithmetic that does not synthesize cheaply. You do not need any of them in this phase.
No <code>===</code> , <code>!==</code> , <code>==?</code> , <code>!=?</code>	Not synthesizable. Use <code>==</code> and <code>!=</code> .
Shifts only by a constant	<code>x << 3</code> is fine; <code>x << i_op2[4:0]</code> is <i>rejected</i> .
No <code>if/else</code> in combinational logic	<code>if/else</code> is only legal inside a clocked <code>always @(posedge ...)</code> block. For combinational logic use a continuous <code>assign</code> with the <code>?:</code> operator, or a <code>case</code> inside <code>always @(*)</code> (valid for generate blocks on parameters and sequential logic only), including your forwarding-select muxes and hazard-detection logic.
No procedural loops	<code>for</code> , <code>while</code> , <code>repeat</code> and <code>do/while</code> statements are rejected. To build something repetitive, use a <code>generate</code> loop, or just write the instances out.
No delays or waits	<code>#5</code> , <code>wait</code> , <code>disable</code> and friends are simulation constructs.
No system tasks	<code>\$display</code> , <code>\$finish</code> , <code>\$stop</code> , <code>\$system</code> , <code>\$readmemh</code> / <code>\$readmemb</code> , <code>\$writememh</code> / <code>\$writememb</code> and all file I/O (<code>\$fopen</code> , <code>\$fwrite</code> , ...) are not allowed in your final submission. Only constant functions such as <code>\$clog2</code> are allowed.
No <code>'include</code>	Not needed for importing other files.
Single clock, synchronous reset	<code>always @(posedge i_clk)</code> with <code>if (i_rst) ... else ...</code> inside it. No second signal in the sensitivity list. This applies to every pipeline register you add, not just the ones Phase 3 already had.

4 Pipelined CPU

In phase 3, you wired your Phase 2 blocks into a single-cycle CPU that fetches, decodes, executes and writes back one instruction per clock cycle. In this phase, you will restructure that same datapath into a multi-stage pipeline. Each stage of the pipeline works on a different instruction at a time.

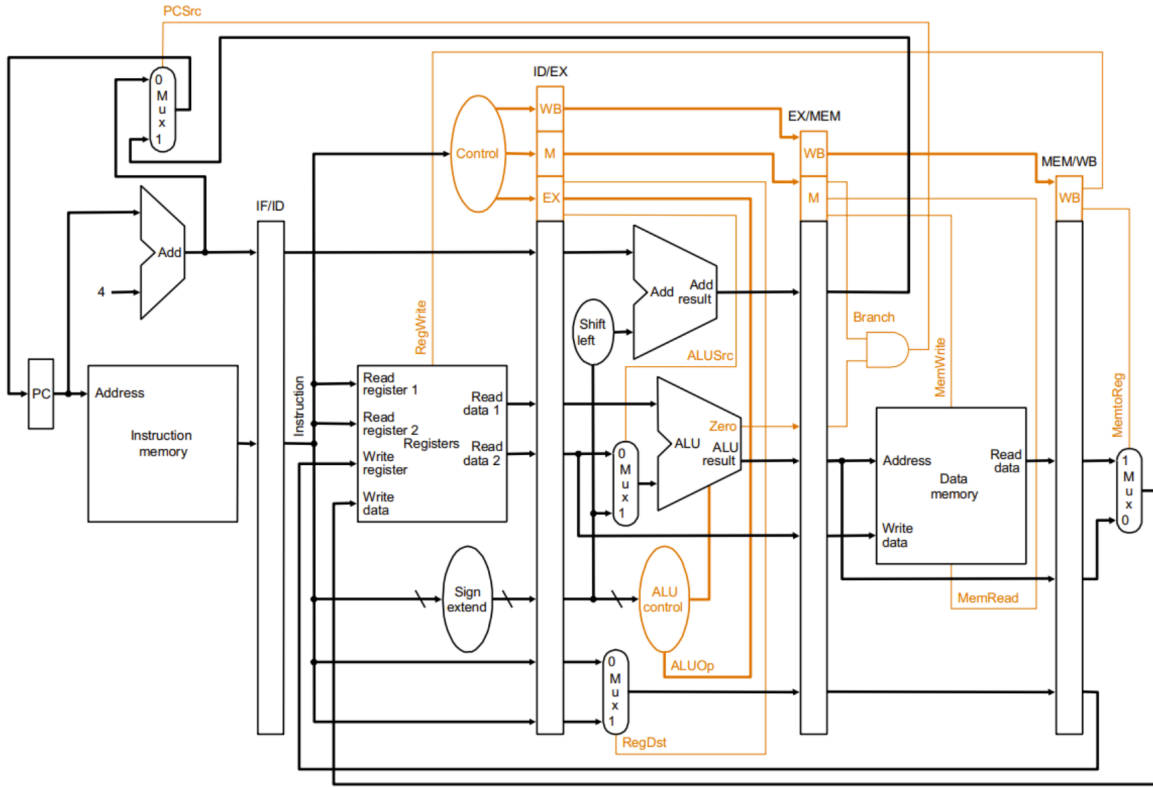


Figure 1: 5-stage pipeline datapath. Stages are separated by register groups (IF, ID, EX, MEM, WB).

4.1 Pipeline regions

The pipeline is divided into five stages, each separated by a set of registers that hold the state of the instruction as it moves through the pipeline.

4.1.1 Instruction Fetch (IF)

The IF stage is responsible for fetching the instruction from memory. It uses the program counter (PC) to determine the address of the instruction to fetch. The fetched instruction is then passed to the next stage. The IF stage also calculated the next PC, acting as a simple branch prediction by assuming branches are not taken.

4.1.2 Instruction Decode (ID)

The ID stage decodes the fetched instruction and reads the necessary registers from the register file. It also generates control signals for the subsequent stages based on the instruction type and

generates immediate values.

4.1.3 Execute (EX)

The Execute stage holds the ALU and branch computation logic.

Note

Branches must be resolved in the EX stage, and then update the PC in the next cycle. Designs that compute the branch target earlier than EX will be rejected. If you follow the diagram above, then your design will be correct.

4.1.4 Memory Access (MEM)

The MEM stage is responsible for accessing data memory for load and store instructions. It uses the address computed in the EX stage to read from or write to memory.

4.1.5 Write Back (WB)

The WB stage writes the result of the instruction back to the register file. This could be the result from the ALU, a value loaded from memory, or a computed immediate value.

4.2 Hazard Detection and Forwarding

Pipeline processors introduce issues in the datapath that can cause incorrect execution if not handled properly. Your processor will need to handle these hazards to ensure correct execution of instructions.

4.2.1 Data Hazards

Data hazards occur when an instruction depends on the result of a previous instruction that has not yet completed its execution. The table below lists the data hazards you need to consider.

Hazard	Dependency	Example	Hazard logic
RAR	Read after read. Two instructions read the same register.	add x7, x5, x6 add x9, x5, x0	No hazard. Read after read causes no pipeline dependencies.
RAW	Read after write. An instruction reads from a previously written register.	add x7, x5, x6 add x8, x7, x0	Hazard. Registers are updated in the write-back stage. So if the read comes within a few cycles after, it will read stale data.
WAR	Write after read. An instruction writes to a register previously read.	add x8, x7, x0 addi x7, x0, 1	No hazard. The register is read before, then gets overwritten.
WAW	Write after write. Two instructions write to the same register.	add x7, x5, x6 addi x7, x0, 1	No hazard. The second instruction overwrites the first.
Load-use	A load memory instruction followed by an instruction that uses the loaded value.	lw x12, 4(x11) add x13, x12, x0	Hazard. The load instruction retrieves its value in the MEM stage, but the use instruction is already in the pipeline reading stale data.
Load-store	The same as Load-use, where the use is a store instruction.	lw x12, 4(x11) sw x12, 8(x10)	Hazard. The load instruction retrieves its value in the MEM stage, but the store instruction is already in the pipeline reading stale data.

4.2.2 Control Hazards

Control hazards occur when the pipeline makes wrong decisions on branch predictions. When a branch instruction is incorrectly predicted, the instructions that were fetched and decoded after the branch need to be flushed from the pipeline.

Note

Your processor must assume that branches are not taken (i.e., the next instruction is always at $PC + 4$) until the branch is resolved in the EX stage. If the branch is taken, you must flush the instructions that were fetched and decoded after the branch, then redirect the PC to the branch target address.

4.2.3 False Dependencies

False dependencies occur when the pipeline detects a dependency that does not actually exist. This can happen from immediate values that align with register bitfields to create false dependencies.

Breakdown

```
add x1, x0, x2
```

```
addi x3, x0, 1.
```

The second instruction has no RAW dependency on the first instruction, but if the immediate value of 1 is encoded in a way that overlaps with the register bitfield of x1, it may create a false dependency. Your processor must correctly identify and handle these cases to avoid unnecessary stalls or flushes.

4.2.4 Stalls and Flushes

When a hazard is detected, your processor must either stall the pipeline or flush the instructions that are in the pipeline.

- **Stalls** are used to delay the execution of instructions until the hazard is resolved. This is typically done by inserting NOPs (no operation) into the pipeline.
- **Flushes** are used to remove instructions from the pipeline that were fetched or decoded based on incorrect assumptions (e.g., branch predictions). This is done by flushing the registers that contain the false instructions.

Note

For NOP instructions, RISC-V uses `addi x0, x0, 0`. Your processor should insert the same instruction when NOP instructions are needed.

4.2.5 Forwarding Logic

Some hazards can be resolved by forwarding the result to the pipeline stage that needs it, rather than stalling the pipeline.

Note

Forwarding is from the output of a stage to the input of a previous stage. For example, an $EX \rightarrow EX$ forwarding path is from the EX/MEM registers to the ID/EX registers.

Hazard	Dependency	Example	Forwarding
RAW	Read after write. An instruction reads from a previously written register.	<code>add x7, x5, x6</code> <code>add x8, x7, x0</code>	Hazard. $EX \rightarrow EX$.
Load-use	A load memory instruction followed by an instruction that uses the loaded value.	<code>lw x5, 4(x4)</code> <code>add x6, x5, x0</code>	Hazard. $MEM \rightarrow EX$.
Load-store	The same as Load-use, where the use is a store instruction.	<code>lw x12, 4(x11)</code> <code>sw x12, 8(x10)</code>	Hazard. $MEM \rightarrow MEM$.

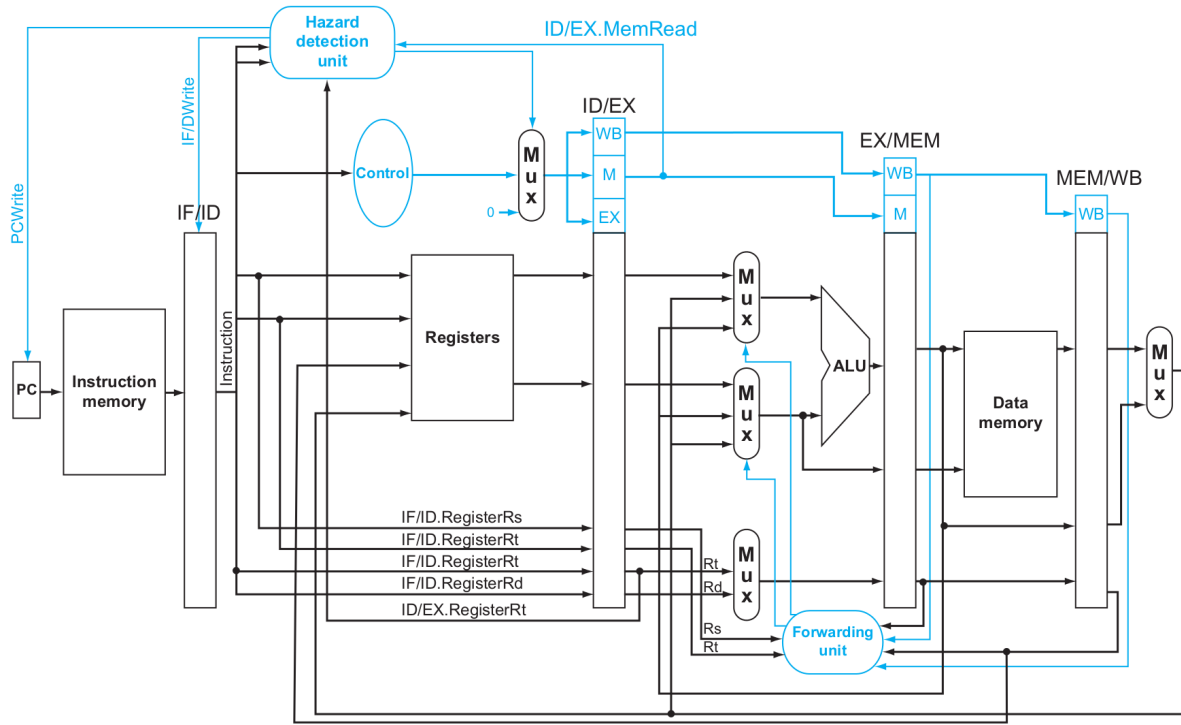


Figure 2: 5-stage pipeline datapath with forwarding logic and hazard detection unit.

Figure 2 shows the forwarding logic and hazard detection unit in the pipeline. The forwarding paths and hazard signals are shown in blue. If your design follows the diagram above, then your design will be correct.

4.3 Memory regions

Phase 3 was graded only at `RESET_ADDR = 0`. Starting with Phase 4, instructions will start with address `0x00400000`. Set your `RESET_ADDR` parameter to that value. This will be reflected in the `hart.v` skeleton.

4.4 Register File

In Phase 4, your register file should be instantiated with `BYPASS_EN = 1`. This allows the register file to return the most recent value of a register, even if it has not yet been written back to the register file. This is necessary for forwarding to work correctly.

4.5 Retire interface

These outputs are not part of RV32I. They exist for the testbench to validate your design and its functionality. Set each signal appropriately every cycle. **Unlike Phase 3**, `o_retire_valid` is not guaranteed high every cycle: fill latency, stalls, and flushes can all leave it low, and none of that is a defect.

Port	Width	Meaning
<code>o_retire_valid</code>	1	An instruction retired this cycle.
<code>o_retire_inst</code>	[31:0]	The raw fetched instruction word.
<code>o_retire_trap</code>	1	Illegal encoding, or a misaligned access.
<code>o_retire_halt</code>	1	The instruction is <code>ebreak</code> .
<code>o_retire_rs1_raddr</code>	[4:0]	Source register 1, 5'd0 if not read.
<code>o_retire_rs1_rdata</code>	[31:0]	Unlike Phase 3, the architecturally correct value rs1 used, reflecting any forwarding , not necessarily the bare register file output.
<code>o_retire_rs2_raddr</code>	[4:0]	Source register 2, 5'd0 if not read.
<code>o_retire_rs2_rdata</code>	[31:0]	The architecturally correct value rs2 used, reflecting any forwarding.
<code>o_retire_rd_waddr</code>	[4:0]	Destination register, 5'd0 if no register is written.
<code>o_retire_rd_wdata</code>	[31:0]	Value written to rd.
<code>o_retire_pc</code>	[31:0]	PC this instruction was fetched from.
<code>o_retire_next_pc</code>	[31:0]	PC + 4, or the branch or jump target when taken.
New for Phase 4: the retiring instruction's own memory op, mirroring the live <code>o_dmem*/i_dmem_rdata</code> ports one-for-one, but timed to the writeback cycle:		
<code>o_retire_dmem_addr</code>	[31:0]	Mirrors <code>o_dmem_addr</code> .
<code>o_retire_dmem_ren</code>	1	Mirrors <code>o_dmem_ren</code> . Never high with <code>o_retire_dmem_wen</code> .
<code>o_retire_dmem_wen</code>	1	Mirrors <code>o_dmem_wen</code> .
<code>o_retire_dmem_mask</code>	[3:0]	Mirrors <code>o_dmem_mask</code> .
<code>o_retire_dmem_wdata</code>	[31:0]	Mirrors <code>o_dmem_wdata</code> . Only the masked lanes matter, same don't-care as the live port.
<code>o_retire_dmem_rdata</code>	[31:0]	Mirrors <code>i_dmem_rdata</code> , the full word actually read at the time of the access.

Note

Every field above is checked on every retiring instruction. Make sure to handle these signals every cycle.

4.5.1 Traps

Traps never redirect control flow. An illegal encoding or a misaligned load/store is a trap. The trapped instruction still retires with `o_retire_trap` set, and execution continues at `pc+4` as if nothing happened. What a trap does change is that the instruction must have no side effects:

- No register write. `o_retire_rd_waddr` is `5'd0`.
- No memory access. Neither `o_dmem_ren` nor `o_dmem_wen` may go high for it, so the retire-side mirrors stay low too.
- For an illegal encoding, `o_retire_rs1_raddr` and `o_retire_rs2_raddr` are also `5'd0`, since nothing was decoded.
- For a misaligned access, the instruction decoded normally, so `rs1_raddr/rs2_raddr` and their `rdata` report the registers that were actually read. A byte access is never misaligned.

In Phase 3 this was easy, as the whole instruction happened in one cycle, so a single `trap` wire could block the register write and the memory enables at once. In a pipeline those side effects happen in later stages, the memory access in MEM and the register write in WB. **Unlike Phase 3**, you therefore have to carry the trap bit through the pipeline registers with the instruction and check it again in every stage that has something to suppress.

`o_retire_halt` is separate from trapping. It is set only when the retiring instruction is `ebreak`, and the program ends on that cycle.

5 Testing Your Work

There is no autograder in this repository. Verifying that `hart.v` runs real programs correctly is part of the assignment. Two independent instruction traces are provided in `phase_4/source/traces/`, with 3 expected output files.

Test	Instruction Trace File	Output Trace File
Pipeline with no hazards	<code>no_hazard_program.hex</code>	<code>no_hazard.trace</code>
Pipeline with hazards	<code>hazard_program.hex</code>	<code>hazard_no_fwd.trace</code>
Pipeline with hazards and forwarding	<code>hazard_program.hex</code>	<code>hazard_fwd.trace</code>

To see the structure of the output trace files, reference `no_hazard.trace`.